

```

// Header file for Network, the abstract base class for neUROn2++ neural networks
//   which use some form of iterative training method.
//   All Networks have weights in some form which may be randomized,
//   a feed forward method, a training method, and a learning rate

#include "stdafx.h" // For MSVC, must be first!

#include "network.h"

// Default constructor
Network::Network()
{
    builtFlag = false; // the Network object hasn't yet been built
    weightsSetFlag = false; // weights not set yet

    // Good ideas for initial values
    randomLimit = 0.5; // a good initial random limit
    biasFlag = true; // Networks should generally have bias nodes
    batchEpochFlag = true; // Networks should generally use batch/epoch training
    weightDecayFlag = true; // Networks should generally use weight decay
    decay = 5e-5; // lambda = 1/(2*sigma^2) where sigma = 100

    // Values that really should be set in the driver
    // The learning rate should be set in the driver to be 1 or 1/N depending on
    //   if training is off-line (batch/epoch) or on-line
    eta = 0.05;

    // Start with canonical backprop
    trainingType = 0;
}

// Default destructor
Network::~Network() { }

// Copy constructor
Network::Network( const Network& rhs )
{
    Network::copy( rhs ); // use the copy utility
}

// Overloaded = operator
Network& Network::operator= ( const Network& rhs )
{
    if ( &rhs != this ) // check for self-assignment
        Network::copy( rhs ); // use the copy utility

    return *this; // enables A = B = C
}

// Copy utility
void Network::copy( const Network& rhs )
{
    Iterative::copy( rhs ); // call immediate base object copy
    output = rhs.output;
    xs = rhs.xs;
    o = rhs.o;
    x = rhs.x;
    randomLimit = rhs.randomLimit;
    eta = rhs.eta;
    decay = rhs.decay;
    decayTerm = rhs.decayTerm;
    regularizer = rhs.regularizer;
    weightsSetFlag = rhs.weightsSetFlag;
    biasFlag = rhs.biasFlag;
    batchEpochFlag = rhs.batchEpochFlag;
    weightDecayFlag = rhs.weightDecayFlag;
    trainingType = rhs.trainingType;
    stackG = rhs.stackG;
}

```

```

    lastF = rhs.lastF;
    lastG = rhs.lastG;
}

// Set the learning rate
void Network::setEta( const double n )
{
    assert( n > 0 && n <= 1 );
    eta = n;
}

// Set the weight decay value
void Network::setDecay( const double n )
{
    assert( n >= 0 && n <= 1 );
    decay = n;
}

// Outputs to ostream reporting the accuracy of a Network Model
void Network::reportAccuracy( ostream& outputStream )
{
    unsigned r, // row iterator
    nInput = theData.getInput(); // number of inputs (easier on the eyes)

    double setError = 0, // set error for test & training sets
    accuracy = 0; // classification accuracy for multiple output sets

    if ( theData.getOutput() == 1 ) // accuracy report for 1-output Networks
    {
        if ( theData.trainLoaded() ) // final training set error
        {
            // Iterate through training exemplars
            for ( r = 0; r < Train.rows(); r++ )
            {
                // Forward propagate the training exemplar
                forward( Train, r );

                // Calculate error for single output and add to set error
                errorFunction E( theData.getTrainMatrix()( r, nInput ), o, x, errorType );
                setError += E.value();

                // If discrete, populate TwoSet object for accuracy report
                if ( theData.getDiscrete() && theData.getTrainTwoSet().loaded() )
                    theData.getTrainTwoSet().test( r ) = o;
            }

            setError /= Train.rows(); // calculate training set error

            // Send resulting report to output stream
            outputStream << resetiosflags( ios::fixed )
                << setiosflags( ios::scientific ) << setprecision( 6 )
                << "Final " << errorLabel << " error in the training set = "
                << setError << "." << endl;
        }

        if ( theData.testLoaded() ) // final test set error
        {
            setError = 0; // reset set error

            // Iterate through test exemplars
            for ( r = 0; r < Test.rows(); r++ )
            {
                // Forward propagate the test exemplar
                forward( Test, r );

                // Calculate error for single output and add to set error
                errorFunction E( theData.getTestMatrix()( r, nInput ), o, x, errorType );
                setError += E.value();
            }
        }
    }
}

```

```

        // If discrete, populate TwoSet object for accuracy report
        if ( theData.getDiscrete() && theData.getTestTwoSet().loaded() )
            theData.getTestTwoSet().test( r ) = o;
    }

    setError /= Test.rows(); // calculate test set error

    // Send resulting report to output stream
    outputStream << resetiosflags( ios::fixed )
        << setiosflags( ios::scientific ) << setprecision( 6 )
        << "Final " << errorLabel << " error in the test set = "
        << setError << "." << endl;
}

if ( theData.getDiscrete() ) // output TwoSet metrics report if discrete
    theData.metricsReport( outputStream );
}

else // multiple outputs
{
    if ( theData.trainLoaded() )
    {
        setError = 0; // reset set error

        // Iterate through training exemplars
        for ( r = 0; r < Train.rows(); r++ )
        {
            // Forward propagate the training exemplar
            forward( Train, r );

            // Calculate error for multiple output and add to set error
            errorFunction E( TrainOutput.row( r ), output, xs, errorType );
            setError += E.value();
        }

        setError /= Train.rows(); // calculate training set error

        // Send resulting report to output stream
        outputStream << resetiosflags( ios::fixed )
            << setiosflags( ios::scientific ) << setprecision( 6 )
            << "Final " << errorLabel << " error in the training set = "
            << setError << "." << endl;

        // If discrete, report final accuracy
        if ( theData.getDiscrete() )
        {
            accuracy = 0; // reset the accuracy

            for ( r = 0; r < Train.rows(); r++ ) // iterate through Train Matrix rows
            {
                forward( Train, r ); // calculate the network's output
                // Make output discrete
                func( output, threshold( theData.getThreshold() ), output );
                if ( output == TrainOutput.row( r ) ) // compare known to guess
                    accuracy++;
            }

            accuracy /= Train.rows(); // calculate the accuracy

            // Send formatted training classification accuracy to ostream
            outputStream << resetiosflags( ios::scientific )
                << setiosflags( ios::fixed ) << setprecision( 1 )
                << "Final classification accuracy in the training set = "
                << accuracy * 100 << "." << endl;
        }
    }
}

```

```

if ( theData.testLoaded() ) // for test set
{
    setError = 0; // reset set error

    // Iterate through test exemplars
    for ( r = 0; r < Test.rows(); r++ )
    {
        // Forward propagate the test exemplar
        forward( Test, r );

        // Calculate error for multiple output and add to set error
        errorFunction E( TestOutput.row( r ), output, xs, errorType );
        setError += E.value();
    }

    setError /= Test.rows(); // calculate test set error

    // Send resulting report to output stream
    outputStream << resetiosflags( ios::fixed )
        << setiosflags( ios::scientific ) << setprecision( 6 )
        << "Final " << errorLabel << " error in the test set = "
        << setError << "." << endl;

    // If discrete, report final accuracy
    if ( theData.getDiscrete() )
    {
        accuracy = 0; // reset the accuracy

        for ( r = 0; r < Test.rows(); r++ ) // iterate through Test Matrix rows
        {
            forward( Test, r ); // calculate the network's output
            // Make output discrete
            func( output, threshold( theData.getThreshold() ), output );
            if ( output == TrainOutput.row( r ) ) // compare known to guess
                accuracy++;
        }

        accuracy /= Test.rows(); // calculate the accuracy

        // Send formatted training classification accuracy to ostream
        outputStream << resetiosflags( ios::scientific )
            << setiosflags( ios::fixed ) << setprecision( 1 )
            << "Final classification accuracy in the test set = "
            << accuracy * 100 << "." << endl;
    }
}

// Outputs classification accuracies to ostream for Iterative table
// Iterative.cpp has already checked that outputs are discrete
void Network::classAccuracy( ostream& outputStream )
{
    unsigned r; // row iterator

    if ( theData.getOutput() == 1 ) // for 1-output Networks
    {
        // Training TwoSet must exist
        if ( theData.trainLoaded() && theData.getTrainTwoSet().loaded() )
        {
            for ( r = 0; r < Train.rows(); r++ ) // iterate through Train Matrix rows
            {
                forward( Train, r ); // calculate the network's output
                // Set the DataSet training TwoSet member's test member to the output
                theData.getTrainTwoSet().test( r ) = o;
            }

            // Send formatted training classification accuracy to ostream

```

```

        outputStream << resetiosflags( ios::scientific )
        << setiosflags( ios::fixed );
        outputStream << " " << setw( 10 ) << setfill( ' ' ) << setprecision( 1 )
        << theData.getTrainTwoSet().getClassAcc() * 100;
    }

    // Test TwoSet must exist
    if ( theData.testLoaded() && theData.getTestTwoSet().loaded() )
    {
        for ( r = 0; r < Test.rows(); r++ ) // iterate through Test Matrix rows
        {
            forward( Test, r ); // calculate the network's output
            // Set the DataSet test TwoSet member's test member to the output
            theData.getTestTwoSet().test( r ) = o;
        }

        // Send formatted training classification accuracy to ostream
        outputStream << resetiosflags( ios::scientific )
        << setiosflags( ios::fixed );
        outputStream << " " << setw( 10 ) << setfill( ' ' ) << setprecision( 1 )
        << theData.getTestTwoSet().getClassAcc() * 100;
    }
}

else // multiple outputs
{
    double accuracy; // the classification accuracy

    // For the training set
    if ( theData.trainLoaded() )
    {
        accuracy = 0; // reset the accuracy

        for ( r = 0; r < Train.rows(); r++ ) // iterate through Train Matrix rows
        {
            forward( Train, r ); // calculate the network's output
            // Make output discrete
            func( output, threshold( theData.getThreshold() ), output );
            if ( output == TrainOutput.row( r ) ) // compare known to guess
                accuracy++;
        }

        accuracy /= Train.rows(); // calculate the accuracy

        // Send formatted training classification accuracy to ostream
        outputStream << resetiosflags( ios::scientific )
        << setiosflags( ios::fixed );
        outputStream << " " << setw( 10 ) << setfill( ' ' ) << setprecision( 1 )
        << accuracy * 100;
    }

    // For the test set
    if ( theData.testLoaded() )
    {
        accuracy = 0; // reset the accuracy

        for ( r = 0; r < Test.rows(); r++ ) // iterate through Test Matrix rows
        {
            forward( Test, r ); // calculate the network's output
            // Make output discrete
            func( output, threshold( theData.getThreshold() ), output );
            if ( output == TrainOutput.row( r ) ) // compare known to guess
                accuracy++;
        }

        accuracy /= Test.rows(); // calculate the accuracy

        // Send formatted test classification accuracy to ostream
    }
}

```

```

        outputStream << resetiosflags( ios::scientific )
        << setiosflags( ios::fixed );
        outputStream << " " << setw( 10 ) << setfill( ' ' ) << setprecision( 1 )
        << accuracy * 100;
    }
}

// Utility to output Network specific parameters prior to an Iterative run
void Network::runHeader( ostream& outputStream )
{
    // A great place to initialize the actual weight multipliers for decay, as this needs
    // only to be calculated once, right before TrainSet() is called iteratively
    regularizer = decay / 2; // the regularization term lambda
    decayTerm = 1 - ( eta * decay ); // 1 - 2*eta*lambda

    switch ( trainingType ) // the training algorithm
    {
        case 0:
            outputStream << "Training algorithm is canonical backpropagation" << endl;
            break;
        case 1:
            outputStream << "Training algorithm is conjugate gradient descent" << endl;
            break;
        case 2:
            outputStream << "Training algorithm is Shanno" << endl;
            break;
    }

    if ( batchEpochFlag ) // batch/epoch learning
        outputStream << "Training is off-line (batch/epoch is on)" << endl;
    else
        outputStream << "Training is on-line (batch/epoch is off)" << endl;

    outputStream << "Learning rate eta: " << eta << endl; // eta

    if ( weightDecayFlag ) // weight decay
        outputStream << "Weight decay term lambda: " << decay << endl;
    else
        outputStream << "Weight decay is off" << endl;
}

// Returns maximum gradient of this Network object
double Network::getGradMax()
{
    // If stackG is not created in trainSet() (e.g. canonical backprop),
    // then one must be created.
    // NOTE: this implies that all other types of Networks create stackG.
    // If you are making a Network that does not create stackG, you must
    // change this if statement so that it will be packed.
    if ( trainingType == 0 )
        pack();

    return maxabs( stackG );
}

// The master training engine, takes training type and iteration as arguments,
// calls the appropriate training algorithm
void Network::engine( unsigned type, unsigned t )
{
    switch ( type ) // choose training algorithm
    {
        case 1:
            CGD( t ); // conjugate gradient descent
            break;
        case 2:
            shanno( t ); // Shanno algorithm
            break;
    }
}

```

```

    }
}

// Conjugate gradient descent, Golden pp. 221-222
void Network::CGD( unsigned t )
{
    pack(); // creates gradient vector stackG from gradient structure

    if ( ( t == 0 ) || ( t == df() ) ) // Golden, step 1

        //  $\vec{f}(t) = -\vec{g}_t$ 

        // Note that lastF refers to *negative* F in Golden's book,
        // So for the purposes here, return stackG,

        // and remember  $\vec{g}_{t-1}$  for next iteration

        lastG = stackG;

    else // Golden, step 2
    {
        //  $\vec{u}_t = \vec{g}_t - \vec{g}_{t-1}$ 

        vector< double > u = stackG - lastG;

        // Remember  $\vec{g}_{t-1}$  for next iteration

        lastG = stackG;

        // Since lastF refers to *negative* F in Golden's book
        // D will thus have reversed sign (-) from Golden's book
        double D = dotprod( u, lastF );

        if ( D != 0 ) // catch division by zero
        {
            //  $b_t = \frac{\vec{u}_t^T \vec{g}_t}{\vec{u}_t^T \vec{f}(t-1)}$ ,  $b_t$  will thus have reversed sign (-) from Golden's book

            double b = dotprod( u, stackG ) / D;

            //  $\vec{f}(t) = -\vec{g}_t + b_t \vec{f}(t-1)$ 

            // with the sign notes from above
            stackG -= ( lastF * b );
        }
    }

    // stackG was loaded with  $\vec{f}$ 

    lastF = stackG;

    unpack(); // return gradient structure from stackG
}

// Shanno algorithm, Golden pp. 217-218
void Network::shanno( unsigned t )
{
    pack(); // creates gradient vector stackG from gradient structure

    if ( ( t == 0 ) || ( t == df() ) ) // Golden, step 1

        //  $\vec{f}(t) = -\vec{g}_t$ 

        // Note that lastF refers to *negative* F in Golden's book,
        // So for the purposes here, return stackG,

```

```

// and remember  $\vec{g}_{t-1}$  for next iteration
lastG = stackG;
else // Golden, step 2
{
    //  $\vec{u}_t = \vec{g}_t - \vec{g}_{t-1}$ 
    vector< double > u = stackG - lastG;
    // Remember  $\vec{g}_{t-1}$  for next iteration
    lastG = stackG;
    // Since lastF refers to *negative* F in Golden's book
    // D will thus have reversed sign (-) from Golden's book
    double D = dotprod( lastF, u ); // denominator for all coefficients
    if ( D != 0 ) // catch division by zero
    {
        //  $a_t = \frac{\vec{f}(t-1)^T \vec{g}_t}{\vec{f}(t-1)^T \vec{u}_t}$ ,  $a_t$  will thus have same sign as Golden's book
        double a = dotprod( lastF, stackG ) / D;
        //  $b_t = \frac{\vec{u}_t^T \vec{g}_t}{\vec{f}(t-1)^T \vec{u}_t}$ ,  $b_t$  will thus have reversed sign (-) from Golden's book
        double b = dotprod( u, stackG ) / D;
        //  $c_t = \gamma + \frac{|\vec{u}_t|^2}{\vec{f}(t-1)^T \vec{u}_t}$ , 2nd term in  $c_t$  will have reversed sign (-) from Golden's book
        double c = eta - ( squared( u ) / D );
        //  $\vec{f}(t) = -\vec{g}_t + a_t \vec{u}_t + (b_t - c_t a_t) \vec{f}(t-1)$ 
        // with the sign notes from above
        stackG -= ( u * a );
        lastF *= ( b + ( c * a ) );
        stackG -= lastF;
    }
}

// stackG was loaded with  $\vec{f}$ 
lastF = stackG;
unpack(); // return gradient structure from stackG
}

```